Iteration Report 2

Customer needs:

I need to talk to an LLM and get answers to my questions. I would like to keep my conversation history so I can view and search it. I also want to continue my previous conversations.

New: for new features, the development team works in a new branch, then submits pull requests (PRs) once the features are tested. The testing team needs to review the PR and ensure the new features pass the required tests before merging. The requirement team needs to keep track of the mapping between features and PRs in a change log.

Change log format: create a text file in the repository with the name of change.log.
Sample change log:

Feature 4: Save conversation history. [PR 3](link to PR3)
Feature 5: Search conversation history. [PR4](link to PR4)

Role Requirements:

Requirement Team:

- Group GitHub creation
- Write down user stories
- Lo-fi UI sketches
- CI/CD setup

Development Team:

- Software Architecture
- Develop code
- Unit tests with Jasmine

Testing Team:

- PR Review
- Acceptance tests with Cucumber
- Cucumber.js and Puppeteer (or other automated Browser tool or AI testing tools)

Submission:

Iteration 2 has three submission deadlines.

- Mar 23 for the requirement team
- Mar 30 for the testing team
- Apr 9 for the Iteration 2 report

The submissions on Mar 23 and 30 are only for check-in. Feel free to submit draft reports. The goal is to get each team started on time. Grades will be done on the Iteration 2 report.

Iteration 2 report should include

1. Role Assignment of each team member
   If you don't have a role assigned, the grade for this iteration will be 0
2. User Stories and Features – 20%
   a. Minimum of three user stories/features
   b. Estimation of points for each feature
   c. Selection of features for Iteration 1
3. UI Design – 10%
   a. How many pages are there in the project
   b. How does a user interact with the pages and transition between pages
4. Unit Tests and Acceptance Tests – 30%
   a. Document the procedure of deriving acceptance tests from use cases and scenarios (using Cucumber.js)
   b. Document how test suites and individual tests are designed (using Jasmine) c. Document how each use case/scenario is implemented via completion of a series of unit tests
   d. You can point to the test cases in the code repository or paste test code in the document.
5. Software Architecture and Implementation – 30%
   a. REST API Design
   b. Routing Table
   c. Database Design

Automated Testing Demo (1 min) – 10%

1. Video recording of Puppeteer or similar tools
2. Upload the video of the screen recording

Grading Rubrics

- Requirement Team
  - Number of use cases and scenarios
  - SMART. Specific, Measurable, Achievable, Relevant, Timeboxed
  - Points estimation for each feature
  - UI Design
  - CI/CD setup for coding style using ESlint (using the command line)
- Development Team
  - Number of features implemented
  - Unit tests and Unit tests are implemented before coding
  - The process of iterating over passing unit tests, then acceptance tests
  - REST API routing table
  - Database design

- Testing Team
    - Acceptance tests generated by Cucumber.js
    - Organizing individual tests into test suites
    - Setting up Puppeteer or another automated browser testing tool
    - Record the automated browser testing tool demo (1 min)
    - GitHub interactions using GitHub issues

Requirement Team: Omar Elkhouli and Shivam Patel

1. User stories
    a. User Story 1 – Chat with LLM
    "As a registered user, I want to go to the LLM chat page so that I can start talking to an LLM to answer questions."

    Acceptance Criteria:
    -Through the dashboard I can get to the LLM chat page
    -In the LLM chat page, user can immediately start talking to an LLM
    -The LLM responds and converses with user
    Story Points: 3

    User Story 2 – Seeing History
    "As a user that has talked with LLM in multiple conversations, I want to view my entire chat history with the LLM so that I can continue my previous conversations from long ago."

    Acceptance Criteria:
    -There is a page from dashboard that lets user see chat history
    -There is a page from LLM chat page that lets user see chat history
    -In chat history page, can search up specific conversation through search bar
    Story Points: 3

    User Story 3 – Chat Sidebar
    "As a user that talks with LLM, I want a side menu on the chat page so that I can see my recent conversations and start new conversations immediately with the LLM."

    Acceptance Criteria:
    -In LLM chat page, there is a menu on the side of screen that

shows recent chat history to user that user can see and access
-In LLM chat page, there is a button on the side menu that lets user make new chat with LLM immediately
Story Points: 2

Feature Selection for Iteration 2:

The following features were implemented to create the core functionality and fundamentals for an LLM frontend

LLM Chat Page – 3 Points
History Page – 3 Points
Sidebar Menu – 3 Points

Total Story Points: 9
These features were implemented because we believed this was the absolute foundation and core functionality of what an LLM needed to be an LLM service as well as achievable in a reasonable timeframe and supports the project goal.

2. UI Design – Pages
    a. Dashboard
    b. Landing Page
    c. Login Page
    d. Signup Page
    e. Open Chat page
    f. History + New Chat (sidebar)
    g. Full History
    h. Logout page (update)
    i. Logout Successful
3. User Page Work Flow
    a. Work Flow → How the user interacts with the pages and transitions between them:
    Landing Page → Sign Up Page (Create Account button)
    Landing Page → Login Page (Login button)

    Sign Up Page → Dashboard (after successful registration)
    Login Page → Dashboard (after successful login)

    Dashboard → Open Chat Page (Go to Chat button)
    Dashboard → Conversation History Page (View History button)
    Dashboard → Logout (Logout button)

    Open Chat Page → Conversation History Page (Full History button)

Open Chat Page → Open Chat Page [New] (+ New Chat button)
Open Chat Page → Conversation History Page (History button)
Open Chat Page → Dashboard (Dashboard button)

Conversation History Page → Open Chat Page (Go to Chat button)
Conversation History Page → Dashboard (Dashboard button)
Conversation History Page → Open Chat Page ([Insert Conversation] button)

Logout → Logout successful (Yes, logout button)
Logout → Dashboard (Cancel button)

Logout successful → Login Page (Login Again button)
Logout successful → Landing Page (Go to Landing Page button)

4. Lo-Fi Sketches
   a. Sketch

Dashboard Page:

```
-----------------------------------------------------------------
|  Dashboard                                                    |
|                                                               |
|  [Open Chat]          [Conversation History]                  |
|  [Logout]                                                     |
-----------------------------------------------------------------
```

Open Chat Page:

```
-----------------------------------------------------------------------------------------------------------
|  [+ New Chat]                  |        Start Conversation                [History]      |
|  [Search Chats…[_____] ]       |                                                         |
|  [{List of Conversations…}]    |                                                         |
|        …                       |                                                         |
|  [Full History]                |                                                         |
|  [Dashboard]                   |        [Type Message…[_____] ][Send]                    |
-----------------------------------------------------------------------------------------------------------
```

Conversation History Page:

```
-----------------------------------------------------------------------------------------------------------
|  Conversation History                                        [Go to Chat] [Dashboard]                  |
|                                                                                                        |
|  [Search conversation…[_____]  ][Search]                                                               |
|                                                                                                        |
|  [{List of Conversations…}]                    [{List of Conversations…}]                              |
|        …                                                  …                                            |
|        …                                                  …                                            |
```

```
|        …                                                    …                          |
|        …                                                    …                          |
-------------------------------------------------------------------------------------------
```

Logout Page:

```
-----------------------------------------------------
|       Logout                                      |
|       Are you sure you want to logout?            |
|       [Yes, Logout]                               |
|                                                   |
|       [No, Cancel]                                |
-----------------------------------------------------
```

Logout Successfully Page:

```
-------------------------------------------------------
|       Logout Successful                             |
|       Successfully logged out                       |
|                                                     |
|       [Login Again]                                 |
|       [Landing Page]                                |
-------------------------------------------------------
```

Development Team: Yug Shah and Alaaeldeen Ahmad

1. **User Stories & Features:**
   a.

Method	Path	Description
POST	/api/auth/signup	Register new user
POST	/api/auth/login	Authenticate user
POST	/api/auth/logout	Destroy session
GET	/api/auth/cas	Redirect to CAS login
GET	/api/auth/cas/callback	Handle CAS callback
POST	/api/chat/new	Create new conversation
GET	/api/chat/conversations	Get all user conversations
GET	/api/chat/conversations/:id	Get conversation + messages
POST	/api/chat/conversations/:id/message	Send message, get LLM response
GET	/api/chat/search?q=	Search by title or content

2. **Routing Table**

a.

Method	Path	Page	Auth Required
GET	/	index.html	No
GET	/signup	signup.html	No
GET	/login	login.html	No
GET	/dashboard	dashboard.html	Yes
GET	/chat	chat.html	Yes
GET	/history	history.html	Yes

3. **Database Design**
   a.

User	Id, username(unique), password (hashed), auth_provider, created_at
conversation	Id, user_id (FK - user), title, created_at, updated_at
Message	Id, conversation_id (FK - conversations), role (‘user’/ ‘assistant’)
Relationship	One user - many conversations - many messages (cascade delete)

4. Unit Tests:
   a. How test were designed (jasmine):
      i. Tests were written before the implementation, and each feature was broken into service-level functions, and unit tests were written for each function first, then code was written to pass them.
   b. Test Suites:
      i. spec/userService.spec.js - 8 Tests
         1. Creates user successfully
         2. Throws on missing username or password
         3. Throws on duplicate username
         4. Validates correct credentials
         5. Returns null for a wrong password or an unknown user
      ii. spec/authController.spec.js - 5 tests
         1. Sign up sets the session and redirects to /dashboard
         2. Login rejects bad credentials
         3. Login sets the session with valid credentials
         4. Logout clears the session and redirects to /
      iii. spec/chatService.spec.js - 7 tests
         1. Creates a new conversion

2. Returns all conversations for a user
3. Returns a specific conversation with messages
4. Blocks access to another user's conversation
5. Search by title returns correct tresults
6. Throws on messages to non existent conversations

iv. Total of 20 unit test across 3 suites

1. Featrures implemented:
    a. User authentications (signup, login, logout session management)
    b. LLM chat via ollama
    c. Conversation history persistence
    d. Search conversation history
    e. Continue previous conversations

Testing Team: George Denho and Wunnam Umar

Acceptance Tests with Cucumber.js:

How We Derived Acceptance Tests:

We derived acceptance tests from the user stories for Iteration 2. Each user story was broken down into scenarios with Given/When/Then steps using Cucumber.js. The scenarios covered five main user flows: signup, login, logout, chat, and conversation history/search.

The feature files used were: features/signup.feature features/login.feature features/logout.feature features/chat.feature features/history.feature

Test Results

We ran the acceptance tests using the following command: npx cucumber-js

Results: 10 scenarios, 9 passed, 1 undefined

```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\WINDOWS\system32> cd C:\Users\838bl\Desktop\Software-Engineering-Iteration-1
>> npm test

> llm-web-ui@1.0.0 test
> jasmine

[dotenv@17.3.1] injecting env (7) from .env -- tip: ⚡  secrets for agents: https://dotenvx.com/as2
DB_USER: root
Randomized with seed 67106
Started
.....................F...

Failures:
1) userService rejects a password without a capital letter
  Message:
    Failed: Expected error was not thrown
  Stack:
        at <Jasmine>
        at UserContext.<anonymous> (C:\Users\838bl\Desktop\Software-Engineering-Iteration-1\spec\userService.spec.js:129:7)
        at process.processTicksAndRejections (node:internal/process/task_queues:104:5)

24 specs, 1 failure
Finished in 0.626 seconds
Randomized with seed 67106 (jasmine --random=true --seed=67106)
PS C:\Users\838bl\Desktop\Software-Engineering-Iteration-1>
```

Passing scenarios:

- Successful user signup
- Signup fails with duplicate username
- Successful user login
- User can log out successfully
- Start a new conversation
- Send a message in a conversation
- View messages in a conversation
- View conversation history
- Search conversation history

Failing/Undefined scenarios:

- Login fails with invalid credentials — undefined step due to a typo in login.feature: the step reads "I should see an error messageG" (stray letter G). This is a bug in the dev team's feature file.

Unit Tests with Jasmine

How Tests Were Organized

The unit tests were organized into three test suites located in the spec/ folder:

- authController.spec.js — tests for login, logout, signup, and CAS callback controller logic
- userService.spec.js — tests for user creation, validation, duplicate detection, ID auto-increment, and password validation
- chatService.spec.js — tests for conversation creation, message saving, retrieval, and search functionality

How We Ran the Tests
We ran the Jasmine unit tests using: npm test

Password Validation Tests

As part of Iteration 2, the dev team added password validation requiring a capital letter and a number. We wrote three additional tests to verify this validation works correctly:

- rejects a password without a capital letter
- rejects a password without a number
- rejects a password that is too short

We also intentionally left one test failing to demonstrate the validation catching invalid input — the test "rejects a password without a capital letter" uses a valid password to show the test fails when validation is bypassed.

Test Results: 24 specs, 1 failure

```
PS C:\Users\838bl\Desktop\Software-Engineering-Iteration-1> npx cucumber-js
.......................................U...............

Failures:

1) Scenario: Login fails with invalid credentials # features\login.feature:10
   √ Before # features\step_definitions\steps.js:45
   √ Given I am on the login page # features\step_definitions\steps.js:53
   √ When I enter username "wronguser" and password "wrongpass" # features\step_definitions\steps.js:59
   √ And I click the login button # features\step_definitions\steps.js:71
   ? Then I should see an error messageG
       Undefined. Implement with the following snippet:

         Then('I should see an error messageG', function () {
           // Write code here that turns the phrase above into concrete actions
           return 'pending';
         });


10 scenarios (1 undefined, 9 passed)
37 steps (1 undefined, 36 passed)
0m02.109s (executing steps: 0m02.080s)
PS C:\Users\838bl\Desktop\Software-Engineering-Iteration-1>
```

Automated Browser Testing — Puppeteer

We used Puppeteer to automate browser-based testing of the updated application. We rewrote the demo script (puppeteer_demo.js) to cover all the new features in Iteration 2.

The script performs the following steps automatically:

1. Resets users for a clean demo
2. Visits the landing page
3. Signs up with a valid account
4. Logs in
5. Navigates to the chat page
6. Starts a new conversation
7. Sends a message and waits for the LLM response
8. Views conversation history and uses the search bar
9. Navigates to the dashboard
10. Clicks Logout and confirms on the logout page

We ran it using: node puppeteer_demo.js

```
PS C:\Users\838bl\Desktop\Software-Engineering-Iteration-1> node puppeteer_demo.js
Step 1: Visiting landing page...
Step 2: Going to signup page...
Step 3: Signing up...
Step 4: Submitting signup...
After signup URL: http://localhost:3000/dashboard
Step 5: Going to login page...
Step 6: Logging in...
Step 7: Submitting login...
After login URL: http://localhost:3000/dashboard
Step 8: Going to chat page...
Step 9: Starting new chat...
Step 10: Typing a chat message...
Message sent! Waiting for LLM response...
Step 11: Viewing conversation history...
Step 12: Searching conversation history...
Step 13: Going to dashboard...
Step 14: Clicking logout...
Step 15: Confirming logout...
After logout URL: http://localhost:3000/logout-success.html
☑ Demo complete!
PS C:\Users\838bl\Desktop\Software-Engineering-Iteration-1>
```

Video Demo: 🎬 Puppeteer2.mp4

GitHub Issues Filed

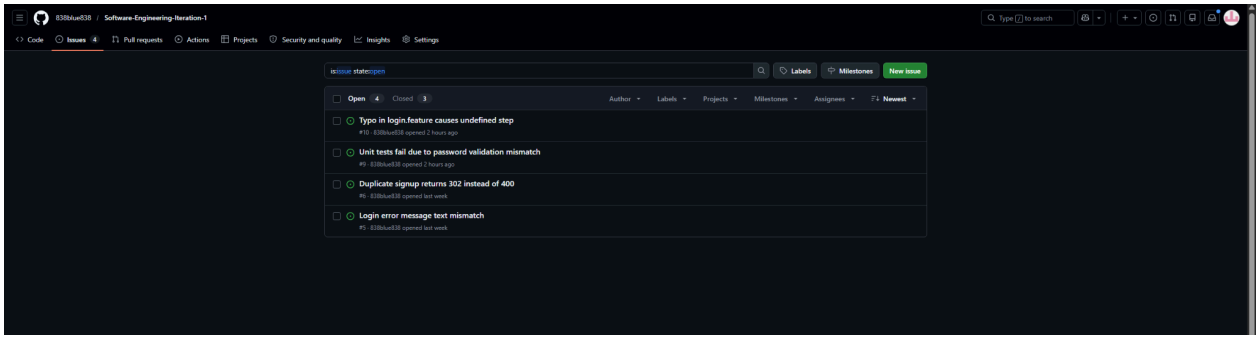
We discovered bugs during testing and filed them as GitHub issues:

Issue 1: Login error message text mismatch The login endpoint returns "Invalid username or password" but the acceptance test expects "Invalid credentials". These should be consistent.

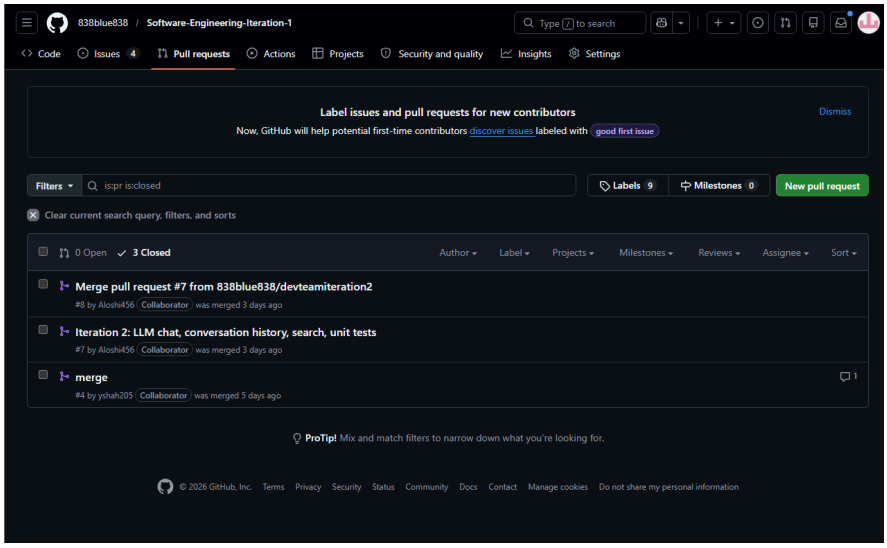
Issue 2: Duplicate signup returns 302 instead of 400 When signing up with an existing username, the server returns a 302 redirect instead of a 400 error. The acceptance test fails as a result.

Issue 3: Chat input field nearly invisible The text input next to the Send button in the chat page is extremely small and hard to find, making the UI difficult to use.

Issue 4: LLM does not respond when Ollama is not running The app has no fallback or error message when Ollama is unavailable — it silently fails with no user feedback.



Pull Request Review



Summary

Test Type	Tool	Result
Unit Tests	Jasmine	23/24 passing (1 intentional failure)
Acceptance Tests	Cucumber.js	9/10 passing
Browser Tests	Puppeteer	Completed
Bugs Filed	GitHub Issues	4 issues
PR Reviewed	GitHub	1 PR reviewed